

CAPSTONE PROJECT

BACHELOR'S DEGREE IN DATA SCIENCE
2025/2026

TECHNICAL REPORT SMART BITES



PREPARED BY

BEATRIZ MARQUES 20231605 | CONSTANÇA PEREIRA DA SILVA 20231720
MARIA INÊS SANTOS 20231630 | MARIANA CALAIS-PEDRO 20231641

TABLE OF CONTENTS

1	EXECUTIVE SUMMARY	2
2	PROBLEM STATEMENT AND OBJECTIVES	3
2.1	PROBLEM STATEMENT	3
2.2	TARGET USERS AND USE CASES.....	3
2.3	PROJECT OBJECTIVES.....	3
2.4	SUCCESS CRITERIA.....	3
2.5	VALUE PROPOSITION.....	3
3	TECHNICAL ARCHITECTURE	4
3.1	SYSTEM ARCHITECTURE OVERVIEW	4
3.2	LAYER ARCHITECTURE	5
3.2.1	UI Layer	5
3.2.2	Service Layer.....	6
3.2.3	Tools Layer	6
3.3	DATA FLOW	7
3.4	EXTERNAL DEPENDENCIES	10
3.4.1	API.....	10
3.4.2	Database	10
4	TECHNOLOGY STACK & JUSTIFICATION	12
4.1	BACKEND.....	12
4.2	FRONTEND	13
4.3	INFRASTRUCTURE (OBSERVABILITY AND DEPLOYMENT).....	13
5	FEATURE SPECIFICATION	15
5.1	IMPLEMENTED FEATURES.....	15
5.2	PLANNED FEATURES	16
5.3	OUT OF SCOPE.....	17
6	DEVELOPMENT ROADMAP.....	18
7	TEAM ORGANIZATION.....	19
7.1	TEAM STRUCTURE AND ROLES	19
7.2	HOW WE COORDINATE WORK.....	19
8	CHALLENGES & RISK MANAGEMENT.....	20
8.1	TECHNICAL CHALLENGES ENCOUNTERED	20
8.2	ANTICIPATED RISKS.....	20
9	NEXT STEPS & OUTSTANDING DECISIONS	21
9.1	ARCHITECTURAL DECISIONS STILL UNDER CONSIDERATION	21
9.2	TECHNICAL QUESTIONS AND UNCERTAINTIES	21
9.3	AREAS REQUIRING CLARIFICATION.....	22
9.4	FUTURE IMPLEMENTATIONS	22
10	CONCLUSION	23

1 EXECUTIVE SUMMARY

University students often aim to eat more balanced meals, yet busy schedules make meal planning and grocery organization challenging. Many repeatedly prepare the same dishes, struggle to find recipes based on ingredients already at home and frequently buy items that eventually go unused. This leads to wasted time, limited dietary variety, and added stress around meal preparation.

SmartBites is an all-in-one intelligent meal-planning platform. Its **AI-driven assistant** that suggests **personalized recipes**, creates **meal plans**, helps users manage their ingredients through a **virtual pantry** and recommends ways to use items efficiently. The system combines AI reasoning, a structured database, and a user-friendly web interface to deliver recommendations that fit users' **lifestyles, and preferences**.

While the platform primarily targets **university students**, it also serves a broader audience. Larger households benefit from **automatic quantity scaling** and coordinate grocery planning. Busy professionals can streamline meal preparation, and beginner cooks gain guided steps to build confidence. *SmartBites* adapts recipes, portion sizes, and planning strategies to support varied lifestyles and reduce waste.

The aim of this technical report is to present a working prototype that includes an AI assistant, recipe search, meal-planning logic, receipt-based ingredient extraction, and database integration. Remaining work focuses on refining UI flows, improving performance, and fully integrating AI insights into structured storage for analytics and personalization.

Keywords: *meal planning, grocery organization, ingredient management, AI-driven assistant, structured database, user-friendly interface, personalization*

2 PROBLEM STATEMENT AND OBJECTIVES

2.1 PROBLEM STATEMENT

University students often struggle to **plan** and prepare meals due to **busy** and **unpredictable schedules**. They may repeatedly **cook the same dishes** or **lack recipes that match available ingredients** and **time-consuming** planning or complex recipes frequently lead to food waste.

2.2 TARGET USERS AND USE CASES

Primary Users: University students who manage their own meals

Use Cases

- **Planning meals** based on available ingredients that align with individual schedules and lifestyles, minimizing food waste through efficient ingredient management
- **Adapting** meals to dietary **preferences**
- Receiving **step-by-step guidance** during meal **preparation**

2.3 PROJECT OBJECTIVES

- Provide **personalized meal plans** tailored to users' lifestyles, ingredients, and time.
- Suggest practical, easy-to-follow recipes with enough variety to prevent repetition.
- **Manage inventory** efficiently with a virtual pantry.
- **Reduce** the **time**, **effort**, and **stress** of meal preparation, shopping, and cooking.

2.4 SUCCESS CRITERIA

- Users can **create a complete meal plan** within minutes and adjust it as needed.
- The system suggests recipes aligned with available ingredients while highlighting any additional items required.

2.5 VALUE PROPOSITION

SmartBites turns meal planning from a stressful task into an easy, enjoyable experience. By helping students eat well, save time, and reduce food waste, its AI-driven recommendations, virtual pantry, and personalized meal plans support nutrition and build confidence in the kitchen.

3 TECHNICAL ARCHITECTURE

This section describes the overall architecture of the *SmartBites* system, its major components, and the interaction patterns that enable the application to manage groceries, generate recipes, and assist users in meal planning.

3.1 SYSTEM ARCHITECTURE OVERVIEW

The system is composed of three main layers:

1. **UI Layer** – user-facing interface and interaction handling.
2. **Service Layer** – core business logic and communication with the *Gemini* model.
3. **Tools Layer** – supporting functions that provide additional processing capabilities or contextual functionality.

The system stores persistent information in a central database, which operates as an external data resource rather than an internal architecture layer.

Authentication is implemented as a cross-cutting module ensuring secure account access and identity validation. Although not part of a dedicated layer, it mediates access to system functionality and to data stored externally.

Below is the system **architecture diagram** that shows the high-level structure of the solution:

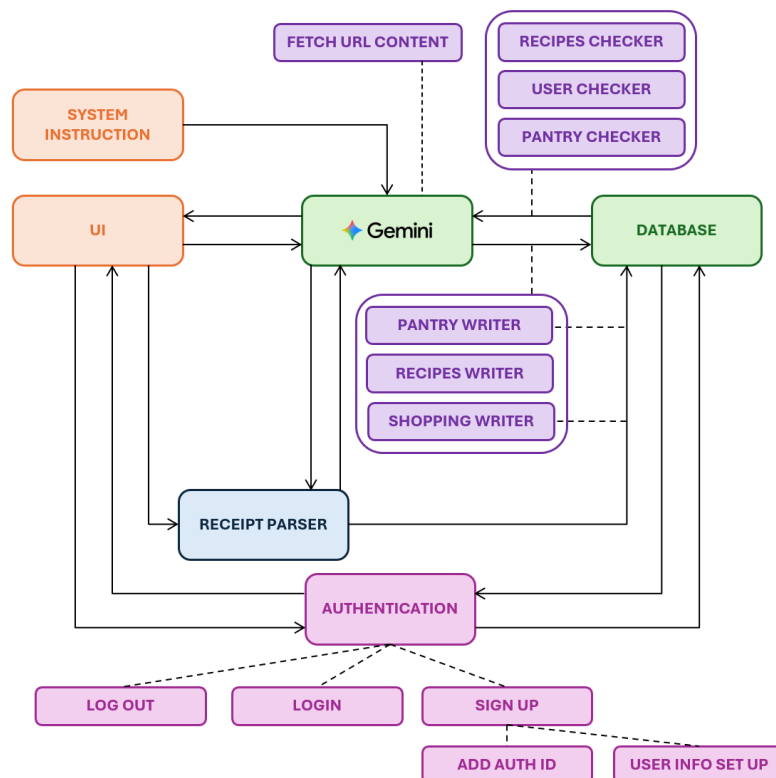


Figure 1 - System Architecture Diagram

SmartBites - Technical Report

The system architecture is centred around the **Gemini core**, which functions as the primary intelligence layer responsible for **reasoning and decision-making**. **User interaction** is initiated through the **User Interface**, while access and **identity management** are controlled by an **Authentication** module that handles login and account setup. Once authenticated, user inputs and instructions are passed to the *Gemini* core for processing.

All persistent **data**, such as recipes, pantry items, and user profiles, is **stored** in the **central database**. The system also incorporates a **Receipt Parser**, enabling **structured extraction of item data from external documents** and routing the results back into the main processing flow.

The **UI communicates with a single service layer**, which mediates access to stored data and invokes the *Gemini* AI model when **intelligent reasoning** or adaptive system responses are required. Supporting this layer, dedicated **tools provide auxiliary functionality such as contextual cooking support**.

Overall, the design clearly separates user interaction, core intelligence, service mediation, and data persistence. This modular structure improves **maintainability**, supports **scalability**, and ensures an **organized system behaviour**.

3.2 LAYER ARCHITECTURE

3.2.1 UI LAYER

The user interface (UI) layer is the point where all the system's components converge and where users interact directly with the application. It handles user input, presents processed information, and ensures a smooth and intuitive experience. Following this way of thought, these were the pages designed:

- **Login and Signup:** Users can log in or create a new account. During signup, they provide their email, password, and additional details like full name and dietary restrictions. Collecting this information upfront allows personalized recommendations, enforces dietary needs, and streamlines meal plan generation, improving overall usability.
- **Chatbot assistant and Receipt Analyser:** Two tabs – one for chatting with the AI to manage meal plans, pantry, or shopping lists, and another for uploading receipts (PDF or image) for automatic database entry.
- **Meal Planner:** Calendar view for past, current, and future weeks; users can search by date and click recipes to see details in a pop-up.
- **Pantry and shopping list:** Two columns showing pantry items with quantities and added dates, and the shopping list.
- **Profile:** View and edit personal information.

Responsibilities

- Ensure an intuitive and seamless user experience, through clear information display.
- Enable smooth interaction with the AI assistant.

Design Rationale

Keeping UI separate **prevents business logic** from **leaking into presentation code** and **allows future replacement** (e.g., migrating from web UI to mobile app). This layer communicates primarily with the services layer to trigger application actions and retrieve processed data. Multiple pages were created in order to have everything separate and more organized, leading to a **better user understanding and interaction**.

3.2.2 SERVICE LAYER

The Service Layer forms the **core of the application**. It encapsulates the domain logic, orchestrates application workflows, and mediates interactions between the **user interface**, the **database**, and the **Gemini reasoning engine**.

Responsibilities

- Orchestrates **application workflows** to ensure smooth operation across modules.
- **Coordinates AI reasoning, content extraction, decision support, and generation tasks** through structured **tool calls**.
- Ensures **consistent** behaviour by following the **same system instructions**.
- Handles incoming **user intent and translates it into deterministic operations**.

Design Rationale

Offloading **NLP** and **reasoning tasks to Gemini** dramatically **reduces backend complexity** and improves functionality. The domain-driven, modular layout keeps modules cohesive, avoids duplicated logic, simplifies maintenance and supports scalability.

Integration

Gemini interacts with the application by:

- **Using** defined **tool** functions to **query** or **update** domain data, **enhance decision-making** and optimize **data interpretation**.
- Operating with a **consistent instruction file** to enforce predictable behaviour
- Maintaining conversational state while the backend remains deterministic and stateless.

This design ensures a clean separation between reasoning (*Gemini*) and persistence (database tools), while allowing the UI to remain lightweight.

3.2.3 TOOLS LAYER

This layer provides **reusable independent modules with a single-responsibility that support the Service Layer**, which **promotes code reuse, reduces redundancy, and keeps domain services and testing simple**.

SmartBites - Technical Report

Pantry Checker	Retrieves a user's pantry items from the database by generating a clear, human-readable summary of ingredients and quantities, ensuring consistent formatting for further AI Service processing.
Pantry Writer	Updates the virtual pantry by inserting entries from receipts or user input, and removing those consumed in approved recipes.
Recipe Checker	Fetches recently cooked recipes' names to avoid similar suggestions.
Recipe Writer	Adds new recipes to the database with user approval, and handles updates or deletions as needed.
Shopping List Writer	Manages shopping list by adding missing ingredients for scheduled meals and updating items from parsed receipts .
User Checker	Retrieves user profile and dietary data , producing clear demographics and preferences for AI use.
Fetch URL Content	Accesses a provided URL and retrieves the plaintext content of the page , such as recipe details, handling errors if the page is inaccessible.

3.3 DATA FLOW

This section comprises example flows in our system architecture.

AUTHENTICATION

Trigger: A new user wants to create an account or an already signed up user wants to login.

Prompt: “New user wants to create an account” or “User wants to login in”

Flow summary:

- SignUp:** The **UI** first **collects the user's login credentials** (a valid email and password). If the input is invalid, the UI prompts the user to correct it until it meets the required format. **Once validated, the credentials are securely sent to the backend and stored in Supabase's auth table**, where the password is automatically encrypted and a **new authenticated user is created**. After the authentication record is successfully created, the UI requests the user's profile information, which is then saved in the users table of the database.

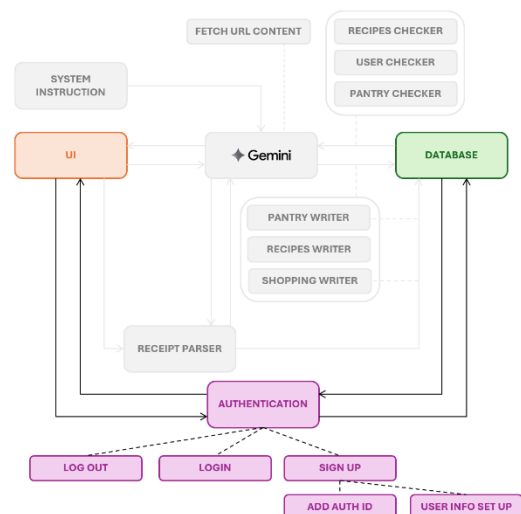


Figure 2 - Authentication Flow

SmartBites - Technical Report

- **Login:** The **UI prompts the user to enter their login credentials** (email and password). These **credentials are then verified against the database**. If the validation fails, the UI continues to request correct information until valid credentials are provided. Once the login is successfully validated, the **UI confirms the successful authentication** and redirects the user to the AI Chatbot page, **granting full access to the web application**.

RECEIPT PROCESSING

Trigger: The user **uploads a receipt photo** through the UI.

Prompt: No conversational **prompt** is involved; this workflow runs **outside the chat interface**.

Flow summary: The **UI forwards the receipt to the Service Layer**, where it is **validated** and passed to Gemini for **text extraction**. It then **converts** the extracted text into **structured item entries**, which are **normalized** and **written to the pantry table**. If any of the purchased items are **present** on the **shopping list**, **quantities are reconciled** and the corresponding shopping entries are removed. The **edible items are inserted into the pantry table** along with their quantities.

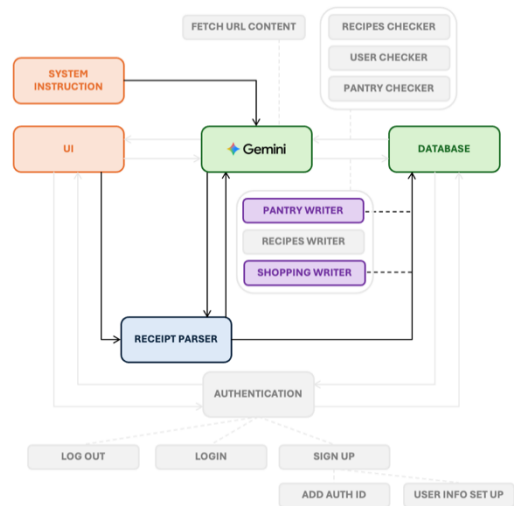


Figure 3 - Receipt Processing

PANTRY, MEAL PLAN & SHOPPING MANAGEMENT

Trigger: User interacts with the system to generate meal plans, add or consume ingredients, or manage shopping lists.

Prompt:

- “Generate a vegetarian 7-day meal plan.”
- “Add 2L of milk to my pantry.”
- “Add 1L of milk to the shopping list.”
- “I had 2 slices of bread at 10:30 today.”

Flow Summary:

- **Meal Plan Generation:** *Gemini* queries the pantry, user preferences, and recent recipes, builds a proposed meal plan and highlights missing ingredients. The UI presents the suggestions, and upon approval the plan is stored while missing items may be added to the shopping list.

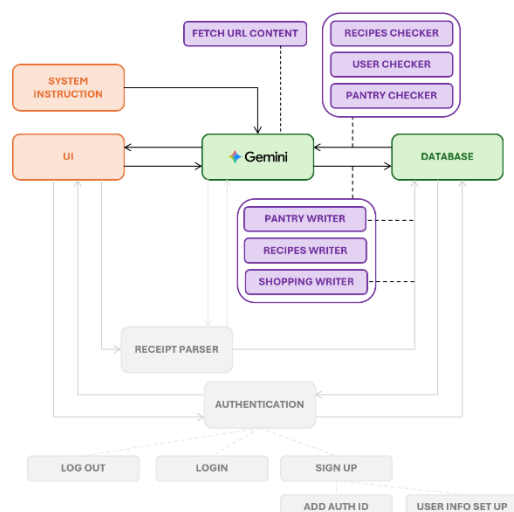


Figure 4 - Database Management Flow

SmartBites - Technical Report

- **Pantry Updates:** *Gemini* extracts the ingredient name and quantity from user input to update pantry entries, and when consumption is reported it adjusts pantry levels, warning if future meals lack ingredients and suggesting replacements when needed.
- **Shopping List Management:** *Gemini* adds or removes items manually or automatically from meal plans, with all changes stored in the database and shown in the UI. When receipts are later processed, the corresponding items are matched and cleared.

All modifications to the pantry, meal plans, and shopping lists are **written to the database according to Gemini's integration**. Gemini, in turn, **relies on this stored information** to generate accurate outputs, creating a **continuous feedback loop** between reasoning and persistence.

COOKING HELPER

Trigger: User submits a cooking-related or culinary technique question.

Prompt: “How do I pan-sear ribeye steak?”

Flow Summary: Upon receiving a query, Gemini provides step-by-step cooking guidance conversationally via the UI, **using its internal reasoning and knowledge base**. The **database remains unchanged**.

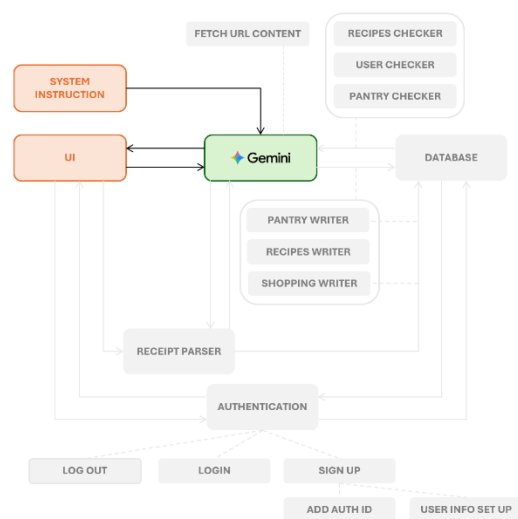


Figure 5 - Cooking Helper Flow

FETCHING USER DATA

Trigger: User requests data from the database, such as pantry contents or meal plan.

Prompt:

- “Show me all the ingredients I currently have in my pantry.”
- “Show me my meal plan for this week.”

Flow Summary: *Gemini* **retrieves the requested data** (pantry items with quantities, or meal plans with recipes) through the tools layer. It then **presents the information in a clear, human-readable** format via the UI. **No database modifications** occur during this process, ensuring accurate view of their stored data.

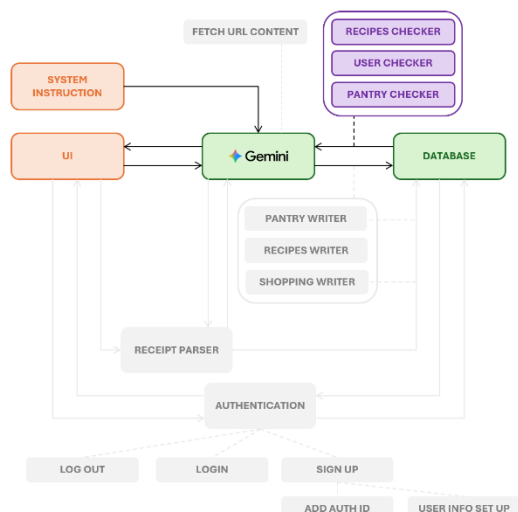


Figure 6 - Fetching User Data Flow

3.4 EXTERNAL DEPENDENCIES

3.4.1 API

SmartBites uses an **internal API as a translation layer between the user and the AI**, mediating tasks such as recipe generation and other AI-driven requests.

3.4.2 DATABASE

A **reliable and structured database is essential for *SmartBites***, since **each user has their own personalised data stored** in the application, including profile information, generated recipes, meal plans, pantry items, and shopping lists. **Without a dedicated persistence layer, the system would not be able to securely store and organise any of this user-specific content.**

To support these needs, *SmartBites* uses a relational SQL database, **Supabase**, chosen because it **offers strong data consistency, well-defined relationships, and robust integrity constraints**, advantages that are particularly important in applications where multiple types of interconnected data must remain accurate and synchronised.

The database serves purely as the persistence component of the system, storing all long-term domain data while remaining separate from the application logic. It is **accessed exclusively through the service layer according to the respective authentication id**, ensuring controlled data flow and business-rule validation.

Database Schema

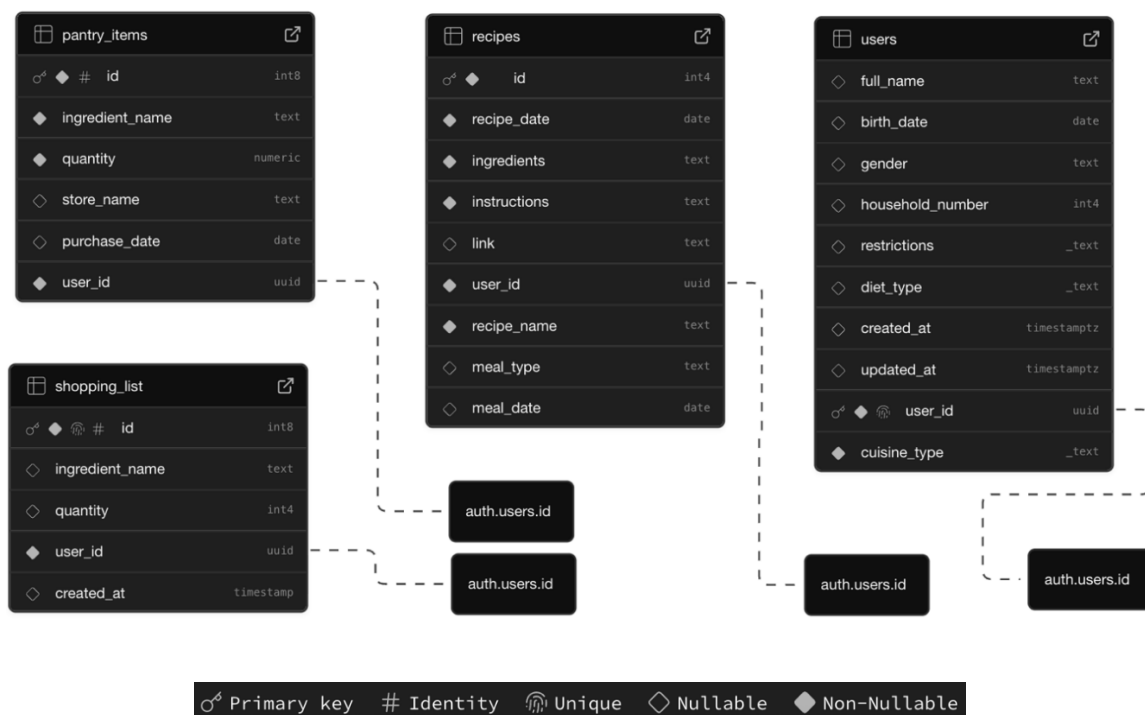


Figure 7 - Database Schema

Authentication & User Linking

The project uses **Supabase Auth** to handle user registration and login. Each table in the database includes a **user_id** column, which stores the authenticated user's **UUID** and references **auth.users.id**. **This establishes a consistent link between the authentication system and all user-specific application data.**

Row Level Based Security

RLS is enabled on every table in the database. **Custom policies** ensure that:

- Users can **only view or modify rows where user_id = auth.users.id**.
- **No user can ever access another user's data.**
- All privacy and access control rules are enforced at the **database layer**, not just the application layer.

This is the core **mechanism that ensures complete data isolation per user.**

Tables Overview

The database consists of four main tables.

Users: Extends *Supabase Auth* with additional profile and preference information, including full name, birth date, gender, default household size, dietary preferences, and favourite recipes.

Shopping_list: Stores items the user adds to their shopping list, alongside with the quantity needed of each item.

Recipes: Stores the information about the recipes that the AI chat bot suggested. Additionally, it stores the date of when the user plans to cook the meal and the type of meal (breakfast, lunch or dinner), serving as the meal-planning component.

Pantry_items: Stores items available in the user's pantry, including the quantity of each item and the date they were added to the database.

4 TECHNOLOGY STACK & JUSTIFICATION

4.1 BACKEND

LLM Provider

The system integrates **Gemini 2.5 Flash**, selected for its combination of high-speed inference, controllable reasoning, and multimodal capabilities. This model supports **real-time** AI functions such as automated meal planning, ingredient reasoning, and personalized recommendations while **minimizing latency and computational cost**. *Gemini 2.5 Flash* offers an excellent balance of performance and efficiency, delivering high-quality results at a lower cost per token than flagship models, making it suitable for high-volume applications. Its low-latency design ensures responsive interactions, while advanced reasoning with a configurable “thinking budget” allows complex tasks to be handled efficiently. With a **context window exceeding one million tokens**, the model can process **long conversations, large documents, or extensive codebases** without truncation. Overall, it provides a **robust, scalable foundation for fast, accurate, and versatile AI** service.

Python Frameworks and Libraries

The backend system is developed in **Python**, chosen for its flexibility, extensive ecosystem, and strong support for data science, AI, and web development tasks. *Python* enables **rapid development, easy debugging, and seamless integration with modern development tools** such as **GitHub** and **Visual Studio Code**, which were used throughout the project lifecycle.

The project leverages a variety of *Python* libraries to support functionality across AI integration, database access, API handling, and web-based interfaces. All project dependencies are documented in the *requirements.txt* file, ensuring reproducibility and ease of deployment. Among these, the most critical libraries include:

- **Google:** Provides access to advanced AI functionalities through tools such as *GenAI*, supporting NLP, content generation, and multimodal data handling.
- **Streamlit:** Used for building an interactive and user-friendly web interface.
- **Langfuse:** Implements observability and tracing of AI-driven operations, enhancing monitoring and debugging of user interactions.
- **Datetime:** Handles date and time operations for meal scheduling, for example.
- **JSON:** Enables serialization and deserialization of structured data for database interactions and API communication.
- **Supabase Python Client:** Provides a direct interface with the *Supabase* database for secure, structured, and efficient data access.

Python’s extensive ecosystem and **compatibility** with these libraries allow the **backend** to efficiently **manage data**, support real-time AI-powered interactions, and **integrate seamlessly with the LLM provider**.

Database

At the start of the development process, **MongoDB** was considered as the database solution. However, because it is a **NoSQL system**, it **introduced limitations for this project's requirements**, particularly regarding structured relationships between data entities and the integration of authentication. These constraints made it **less suitable** for a system that depends heavily on consistent, interconnected, user-specific data.

Supabase was therefore selected as the backend platform. As a **SQL-based solution** built on PostgreSQL, **Supabase supports the creation of a well-defined relational schema**, enabling structured connections between tables and ensuring strong data integrity. Beyond its relational model, **Supabase provides built-in authentication** for secure user sign-up and login, as well as **Row Level Security policies** that guarantee each user can access only their own data. Its scalability and integrated developer tools further contribute to a robust, maintainable, and secure backend infrastructure, making **Supabase an appropriate, user-friendly and reliable fit for the system's needs.**

4.2 FRONTEND

The frontend was developed using **Streamlit**, which was provided as the **primary framework during the course**, ensuring consistency with learning objectives and class demonstrations. **Streamlit enables rapid development of interactive web interfaces without requiring extensive frontend engineering effort.** It offers built-in components for form handling, dynamic rendering, and real-time updates, allowing the system to display recommendations, recipes, and meal plans efficiently.

From a **UI/UX perspective**, **Streamlit** allows the creation of intuitive, **user-centred interfaces** with responsive layouts and **immediate feedback on user actions.** Its components support smooth interaction flows, real-time AI-driven suggestions, minimal cognitive load and an easy deployment phase, making it highly suitable for experimentation and user experiences while keeping implementation complexity low.

4.3 INFRASTRUCTURE (OBSERVABILITY AND DEPLOYMENT)

OBSERVABILITY

To ensure the reliability and transparency of *SmartBites'* AI-driven features, **Langfuse** was integrated as the **observability and tracing layer.** **Langfuse** captures detailed logs of all model interactions, including inputs, outputs, and system responses. This **allows the development team to monitor performance, identify bottlenecks, and analyse how prompts behave** in real use cases. By tracking latency, success rates, and prompt effectiveness, **Langfuse** makes AI behaviour auditable and supports continuous flows' optimization.

DEPLOYMENT PLATFORM

Streamlit Community Cloud was chosen as the deployment platform due to its **simplicity**, suitability for prototype applications, and its alignment with the technologies and practices introduced during the course. Since **Streamlit was the framework used**, deploying on its native cloud platform ensured **consistency with the tools already employed**. Additionally, its strong **integration with GitHub** enables seamless deployment **without the need for server configuration** or domain management, reducing setup time and allowing the focus to remain on the actual application and system development.

5 FEATURE SPECIFICATION

5.1 IMPLEMENTED FEATURES

FEATURE	DESCRIPTION	STATUS	KNOWN ISSUES
User Interface	Integrated with <i>Gemini</i> and the database	Functional	Requires further refinement and user experience testing
Authentication	Ensures consistent user ID usage without data leakage.	Fully functional	No issues reported.
System Instructions	Base system instructions implemented; continuously updated as new features are added.	Functional	Needs ongoing updates as the system evolves.
Ingredient Prioritization	Ingredients that were added earlier are given processing priority to ensure they are used efficiently.	Functional	Continue to be refined.
Cooking Assistant Functionality	Integrated into system instructions to assist users during meal preparation.	Functional	Continue to be refined.
Database	Database fully designed and integrated with the system.	Functional	Continue to be refined in case new columns need to be added.
User Checker Tool	Checks user data for consistency and validity.	Fully functional	No issues reported.
Pantry Checker Tool	Verifies pantry contents.	Fully functional	No issues reported.
Recipe Checker Tool	Returns the recipes' names from a limited number of weeks.	Functional	Requires adjustments.
Pantry Writer Tool	Adapts pantry items.	Stable	Cannot yet remove items/quantities automatically based on approved recipes.
Receipt Parser	Reads a receipt, extracts the edible items, inserts them in the pantry and removes them from the shopping list.	Stable	It is not storing only the edible items.

SmartBites - Technical Report

Recipe Writer Tool	Adapts recipe table.	Functional	Batch processing and simultaneous multi-recipe addition still under development; meal plan editing desired.
Shopping Writer Tool	Adds missing ingredients from approved recipes to shopping list. Removes items from shopping list when receives a receipt.	Stable	Issues with text normalization. Missing the shopping list removal through receipt parsing.

5.2 PLANNED FEATURES

FEATURE	PRIORITY	EXPECTED COMPLEXITY	NOTES
Try to use URL Context and Google Search	Critical	High	Enhance information retrieval without relying on external tools, ensuring that it gives a viable source.
Batch Recipe Processing	Critical	Medium	Allow multiple recipes to be added or edited simultaneously to the database.
Shopping List Removal	Critical	High	Enable automatic updates to shopping list based on parsed receipts.
Update User Data	Critical	Medium	Enables users to edit personal information.
Editable Meal Plan	Important	High	Users can modify the meal plan dynamically.
Text Normalization Refinement	Important	High	Resolve outstanding language standardisation issues when uploading things to database.
Seasonal Finder Integration	Nice-to-have	Low	Integrate seamlessly into the system instructions to suggest adequate recipes.
Reset Password	Nice-to-have	High (due to lack of resources)	Must implement secure password reset functionality.
Sign Up – Age Restriction	Nice-to-have	Low	Allow account creation only for users above a specified age.

5.3 OUT OF SCOPE

This section summarizes ideas removed during the design phase, such as:

- **Nutrition Analysis:** Excluded to maintain focus on core meal planning rather than detailed nutritional tracking.
- **Voice-Controlled Cooking Assistant:** Beyond current scope and timeline; could be introduced in later iterations.
- **Meal Cost Estimation:** Adds complexity without being essential to the initial focus on planning and ingredient management.
- **Recipe Sharing / Social Features:** Social components increase complexity; prioritizing personalized planning instead.
- **Gamification / Cooking Rewards:** Considered non-essential to core objectives; potential feature for future versions.

6 DEVELOPMENT ROADMAP

This roadmap translates the project priorities into a timeboxed, actionable plan for completing the *SmartBites* capstone by December 22, 2025. It focuses on delivering a fully integrated prototype, end-to-end testing, performance tuning, and final documentation.

	Beatriz	Constança	Maria Inês	Mariana
Week 1 (Dec 8 - Dec 14) Focus: Backend consistency, and reliable external retrieval.	Implement URL Context & Google Search retrieval instead of external tool Implement a batch recipe processing (multi-add logic)	Prepare UI for later editable interactions Recipe checker is only giving the recipes from the last X weeks.	Normalization utilities (core reusable modules) when loading things to database Cooking assistant implementation.	Ensure the database implements logic to edit items in the shopping list, by prioritizing oldest pantry ingredients, once a receipt is inserted.
Week 2 (Dec15 - Dec 21) Focus: Improve system usability expand functionality.	Only store edible items in pantry when reading a receipt.	Guarantee that UI is still up-to-date. Ensure consistent user flows.	Integrate Seasonal finder. Enable reset password. Implement sign up data restriction.	Enable database changes in meal plan updates and user data.
Submission Day (Dec 22) Focus: Full Review	Prepare final architectural documentation. Integrate all services for deployment.	Ensure UI compatibility in the deployment environment. Minor UI adjustments if needed.	Final integration checks for utility modules.	Prepare database for deployment. Validate DB migrations and environment variables.
	Review repository, finalize documentation, screenshots, and diagrams. Prepare submission-ready package. Guarantee that system instructions are updated with all system logic.			

7 TEAM ORGANIZATION

7.1 TEAM STRUCTURE AND ROLES

The *SmartBites* development team consists of four members, each responsible for a specific area, working collaboratively in a domain-oriented approach.

Beatriz Marques – Team Leader and AI/Systems Architect

Oversees technical direction and ensures architectural alignment. Main responsibilities:

- Coordinate team planning, prioritization, and decision-making
- Design and maintain the overall system architecture
- Lead AI-related components (prompt design, AI workflows, OCR integration)
- Implement Services Layer with clear UI, database, and AI interfaces

Constança Pereira da Silva – Frontend Lead (UI/UX and Web Interface)

Owns user-facing elements and ensures usability and accessibility. Responsibilities:

- Develop web interface and user interaction flows
- Design responsive and intuitive user experiences
- Ensure consistent communication between UI and backend services

Maria Inês Santos – Tools and Utility Logic Developer

Develops reusable components that support core functionality. Her contributions include:

- Build and maintain utility modules
- Ensure tools integrate correctly with service logic and AI outputs
- Support cross-layer testing and validation

Mariana Calais-Pedro – Database and Persistence Lead

Manages data model and ensures reliable persistence operations. Her responsibilities include:

- Design and implement database schema
- Manage CRUD operations with proper validation and data integrity
- Integrate storage logic into the service layer

7.2 HOW WE COORDINATE WORK

Coordination follows a modular approach with **weekly syncs** across UI, AI, database, and tools, supported by shared documentation. Integration is incremental, validated regularly, and aligned to the overall architecture to avoid conflicts.

8 CHALLENGES & RISK MANAGEMENT

8.1 TECHNICAL CHALLENGES ENCOUNTERED

Managing Cross-Layer Dependencies

There was initial **confusion in distinguishing** which components should be **services** and which should be **tools**. This made it difficult to define a clear and consistent architecture for the project.

Resolution: The team clarified the key differences and restructured significant parts of the project according to this newly established understanding.

User Authentication and Password Reset Mechanism

While implementing authentication was straightforward, adding a secure password-reset flow is introducing additional complexity.

Resolution: **Integration of an email-based reset** mechanism.

8.2 ANTICIPATED RISKS

AI Performance Bottlenecks

Real-time recipe generation and data extraction from receipts may **slow down** the app.

Mitigation: Cache common recipe suggestions and optimize AI prompts to reduce processing time.

Multi-Image Receipt Processing & Deduplication

Long or multiple-receipt images may create **duplicate pantry items**, or **low-quality images** (rotated, dark, incomplete) may **fail to extract the information correctly**.

Mitigation: Deduplicate items using name and price matching, detect repeated bounding boxes, group images by hash or timestamp, and handle low-quality images with preprocessing or user feedback prompts.

9 NEXT STEPS & OUTSTANDING DECISIONS

Despite significant progress, several architectural and design-level decisions remain under evaluation and require structured resolution.

9.1 ARCHITECTURAL DECISIONS STILL UNDER CONSIDERATION

Long-Term Recipe Data Strategy

Three alternative models were considered:

1. Using **only AI-sourced** recipes
2. **Storing scraped** recipes from verified sources as a fallback
3. **Hybrid approach** (AI-generated + curated dataset)

The **hybrid model currently appears most advantageous**, offering scalability, content diversity, and quality assurance, however, **storage implications and licensing constraints remain under review**.

Standardization of Ingredient Models

Current ingredient models must reconcile discrepancies between extracted labels and recipe-generation labels. A key open issue is **handling heterogeneous measurement types**: some ingredients are defined by mass (e.g., grams for rice, flour, sugar), while others are counted as discrete units (e.g., “1 yogurt,” “2 breads”). **Unit-based items introduce ambiguity**, as a single unit may represent different actual quantities (e.g., 1 yogurt could be a 150 g cup or a 1 kg tub). **Canonical naming rules and a standard unit-equivalence mapping are still pending to ensure consistent interpretation across all systems**.

Definition of Prompt Templates for Meal Planning

Current **meal-planning prompts should be optimized to guide users** with all the questions the model needs, like dietary preferences, ingredient substitutions, portion sizes, or default settings, so the user knows exactly what to answer, allowing *SmartBites* to generate meal plans faster and more accurately.

9.2 TECHNICAL QUESTIONS AND UNCERTAINTIES

- ? How much storage is needed to maintain a hybrid dataset of AI-generated and curated recipes?
- ? Are there licensing or copyright issues with storing and using scraped recipes from verified sources?
- ? How can unit-based items (e.g., “1 yogurt”) be standardized to reflect actual quantity across different packaging sizes?
- ? How should prompts be structured to balance completeness and simplicity for the user?

9.3 AREAS REQUIRING CLARIFICATION

- **Storage Requirements:** Precise estimation of storage needed for maintaining a hybrid dataset of AI-generated and curated recipes.
- **Legal Considerations:** Confirmation of licensing or copyright constraints for storing and using scraped recipes from verified sources.
- **Ingredient Standardization:** Definition of canonical naming rules, unit-equivalence mappings, and handling of ambiguous unit-based items (e.g., “1 yogurt”).
- **Prompt Design:** Guidelines for structuring meal-planning prompts to balance completeness with simplicity for the user.

9.4 FUTURE IMPLEMENTATIONS

As mentioned in **Section 5.3**, we identified several innovative ideas that were not developed in the current phase, as they deviated from the main project goals and requirements. These features **remain strong candidates for future iterations** and could significantly enhance the platform:

- **Ingredient Expiration Enhancements:** The system could be extended to incorporate true expiration dates rather than relying on receipt-entry dates, improving accuracy in ingredient prioritization and reducing food waste.
- **Nutrition Analysis:** A full nutritional breakdown feature could be introduced to support users who want detailed insights into the health impact of their meals.
- **Voice-Controlled Cooking Assistant:** Hands-free interaction could be implemented to improve the cooking experience and accessibility during meal preparation.
- **Meal Cost Estimation:** Future versions could include cost predictions for planned meals, helping users manage budgets more effectively.
- **Recipe Sharing / Social Features:** Community-driven recipe sharing and social interaction tools could be added to foster collaboration and user engagement.
- **Gamification / Cooking Rewards:** Reward systems and gamified challenges could be introduced to encourage consistent use and make meal planning more engaging.

10 CONCLUSION

SmartBites provides a comprehensive solution to the challenges of meal planning faced by university students by integrating AI-driven recipe finder, personalized meal plans, and a virtual pantry for efficient ingredient management. The platform streamlines the entire process of planning, shopping, and cooking, reducing effort, minimizing food waste, and supporting users to make informed choices about their meals.

The system's modular, layered architecture ensures maintainability, scalability, and clear separation of concerns across all components. The current prototype already supports intelligent recommendation workflows, multi-layer communication and text extraction. The remaining development focuses primarily on refined integration, interface polish, and validation of consistency across system layers.

Based on current status, the team is confident that all major deliverables will be completed within the established timeline. The system demonstrates strong viability from both a technical and functional perspective, while the architectural foundation allows continuous expansion beyond the current academic scope.

***SmartBites*, therefore presents a robust, realistic, and scalable solution with tangible value, high relevance to real user needs, and clear potential for academic and real-world impact.**